Признаки плохого кода

https://www.codewars.com/kata/reviews/550b304659513854f3000f52/groups/5e10d378bb266200013d5af5

Загадочные имена

- Имя должно описывать цель существования переменной
- Всеми силами избегайте непонимания
- Имена должны быть произносимыми
- Имя должно быть удобно искать
- Стратегия именования должна быть согласованной

Одна из основных особенностей плохого кода — стратегия именования сущностей. Если в команде отсутствуют соглашения об именовании, их следует принять. С ростом приложения правильное именование становится критически важным. Общие принципы именования помогут команде (особенно если в ней есть разработчики разного уровня) находить общий язык и лишний раз не ломать голову в попытках понять\придумать имя для переменной.

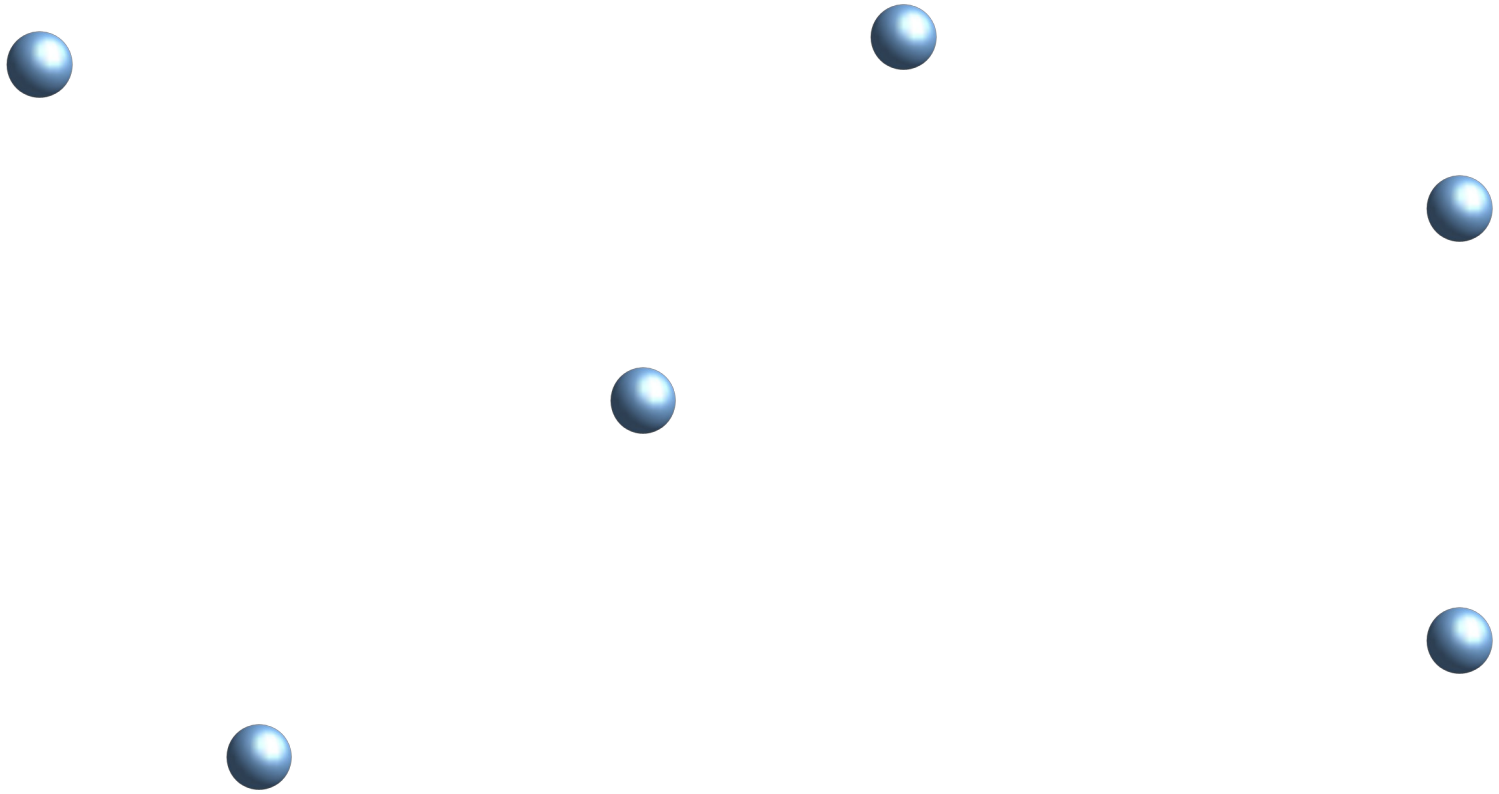
Дублирующийся код

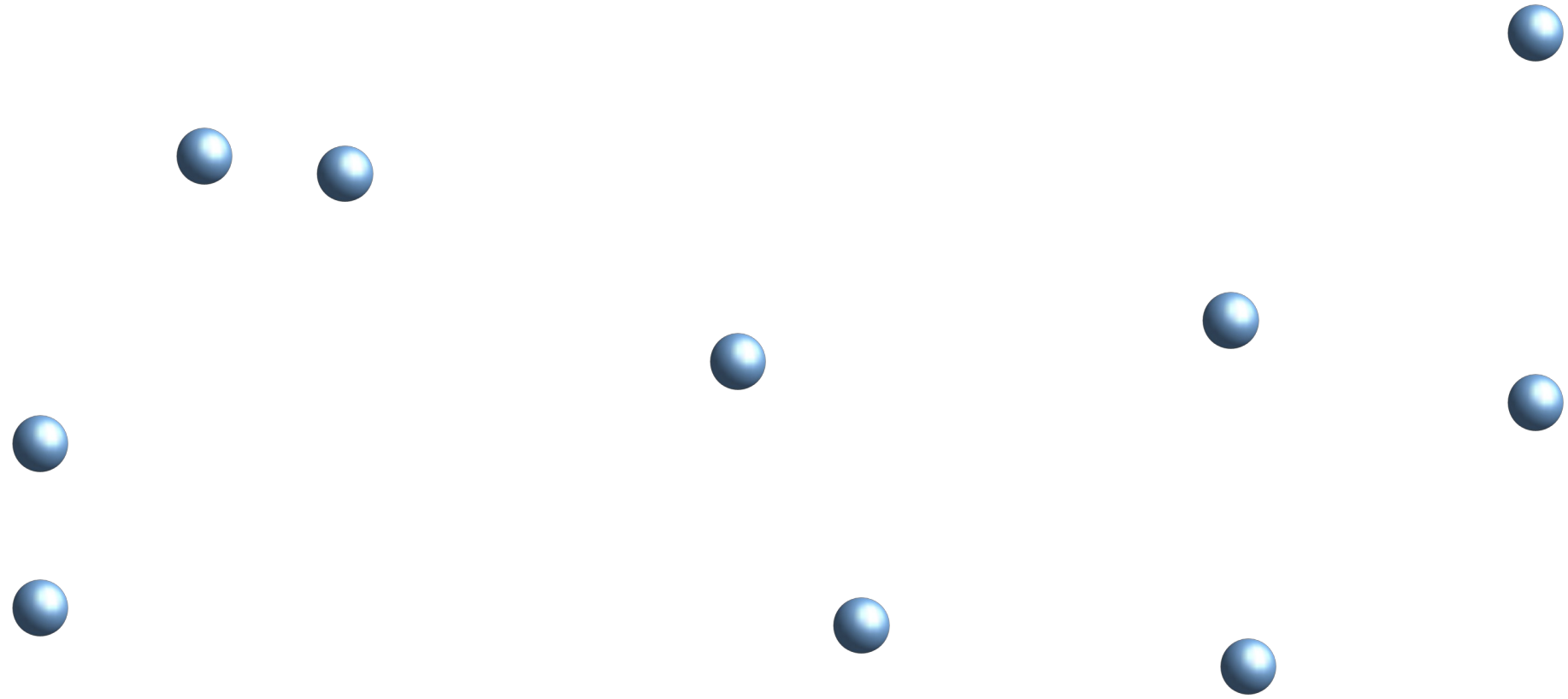
Идентичный код, который разбросан по всему приложению. Он увеличивает сложность поддержки и тестирования системы. Повторяющиеся куски кода являются признаком того, что вам пора задуматься о рефакторинге.

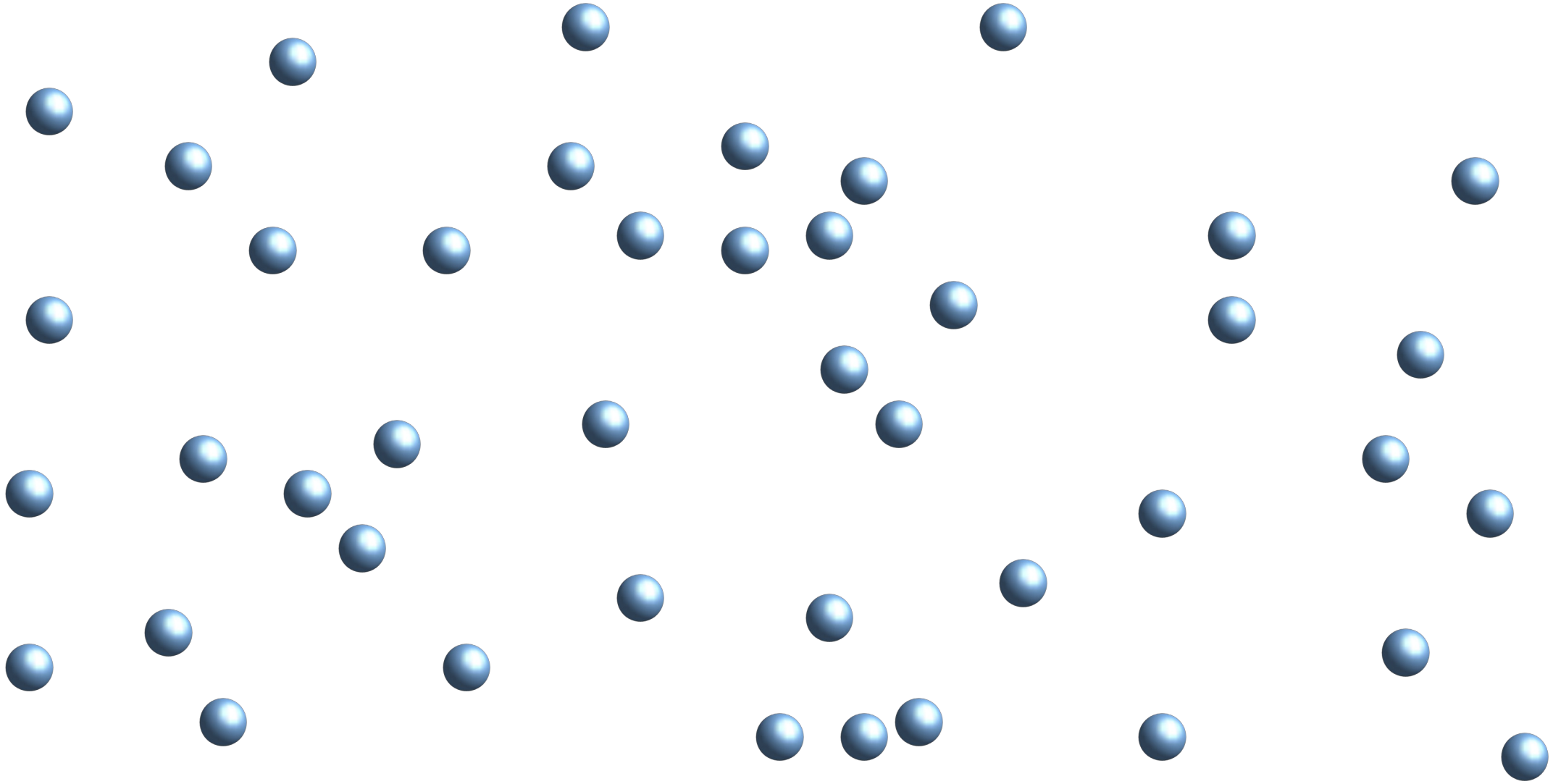
Огромные методы

Слишком большие методы являются источником ошибок и сложны для понимания. Есть правило: функция должна выполнять одну задачу и выполнять её хорошо.









Божественный объект

Так называют огромный класс, который делает слишком много разных вещей. И класс и функция, должен иметь одну цель существования. Поэтому *божественный объект* должен быть разделён на несколько сущностей, каждая из которых имеет только одну задачу.

Избыток параметров

Длинный список параметров усложняет чтение, вызов и тестирование функций. Уменьшение числа параметров позволит вам сократить время на изучение и тестирование кода.

Неуместная сложность

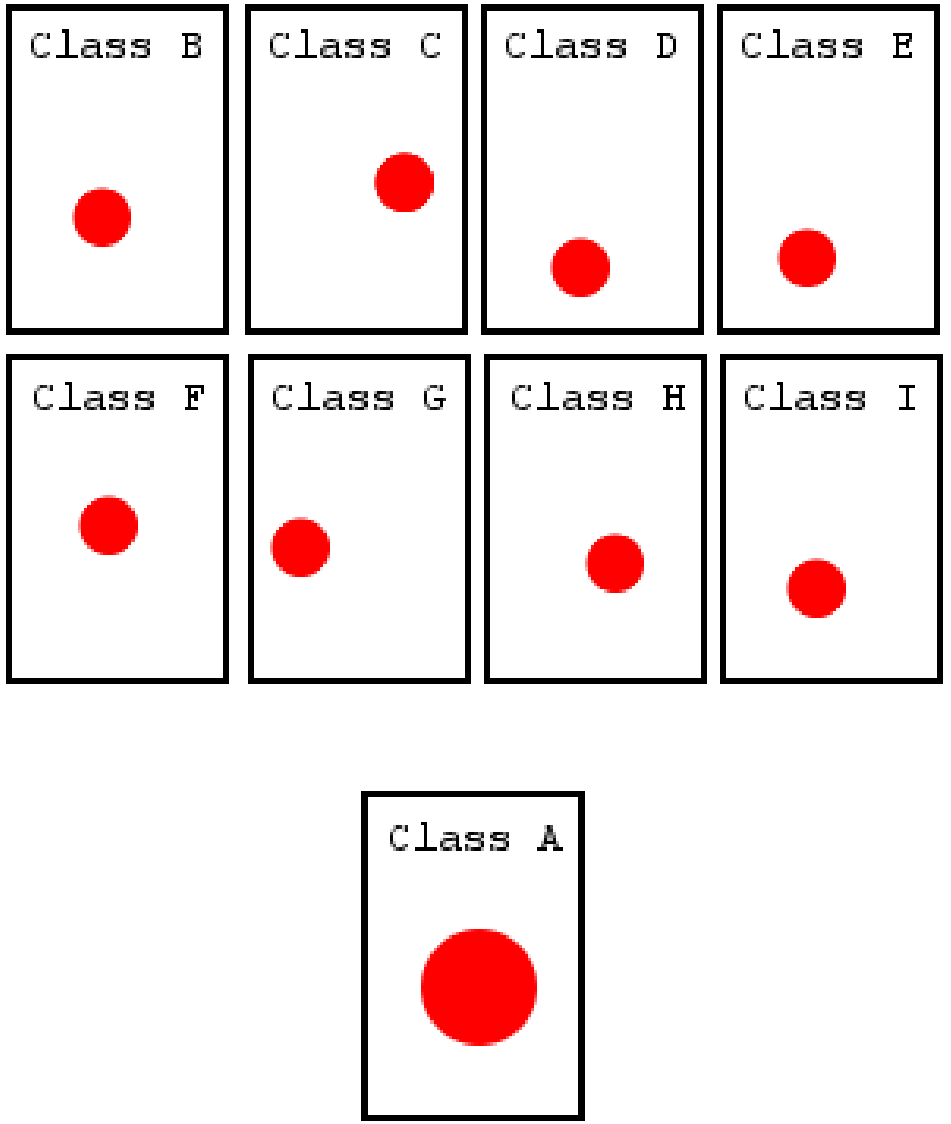
Принудительное использование чрезмерно сложных шаблонов проектирования там, где более простой архитектуры было бы достаточно.

Использование сложных паттернов без необходимости показывает не ваш уровень, а неспособность увидеть картину целиком и избежать излишней сложности.

Хирургия дробовиком

Термин «shotgun surgery» используется для случая, когда одно изменение в коде влечёт за собой множество других изменений.

Изменения в классе A требуют множества незначительных изменений в других классах

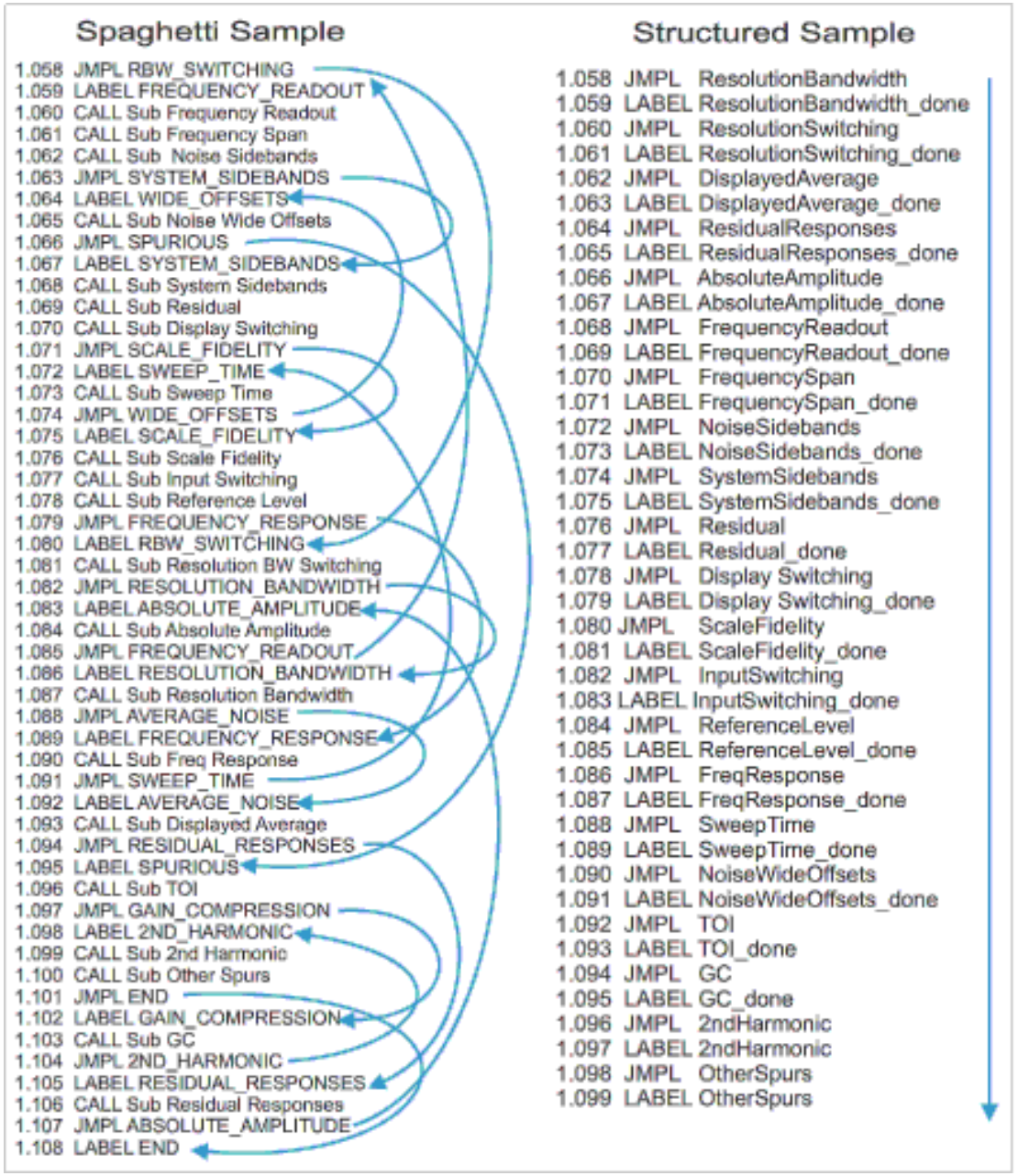


Изменяемость переменных

Код, переменные в котором изменяются непредсказуемо, сложно отлаживать и проводить рефакторинг.

Спагетти-код

Плохо спроектированная, слабо структурированная, запутанная и трудная для понимания программа, содержащая много операторов GOTO, исключений и других конструкций, ухудшающих структурированность.



Фактор автобуса (кирпича)

Метрика, которая показывает хрупкость проекта в случае внезапной потери ключевых сотрудников. Bus Factor измеряет минимальное количество членов команды, внезапное исчезновение которых приведет к остановке проекта из-за отсутствия необходимых знаний и навыков.

Как сделать проект более устойчивым:
1. Создать базу знаний
2. Сформировать кадровый резерв
3. Сделать ключевого сотрудника ментором
4. Поощряйть обмен опытом
5. Развивать команду

